H3: Strings en hun methoden

Labo 1 · week 3

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Strings**: wat is een string, en hoe verschilt die van een `char` of een `int`?
2. **Strings samenvoegen**: met `+` en met stringinterpolatie `$`
3. **Omzetten**: van en naar strings met `Convert`
4. **Functionaliteit van strings**: `Length`, `Substring`, `IndexOf`, `ToUpper`/`ToLower`, `Replace`, `Trim`



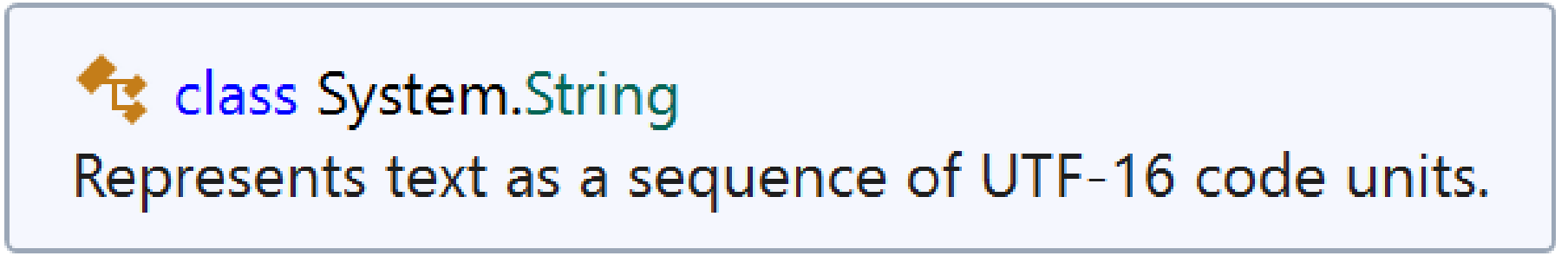
Vandaag gaan we het over strings hebben. Nee, niet die van op het strand.

Strings

Wat is een string?

Een **string** is een reeks van **0, 1 of meerdere** lettertekens.

```
string eenTekst = "Wat een mooi zin";
```



Hover in Visual Studio over het `string`-keyword

Ook een lege string `""` is een string. Net zoals een lege koelkast nog altijd een koelkast is. Alleen triester.

`char` of `string`?

char: één teken, tussen **apostrofs**

```
1  char letter = 'a';
```

string: reeks tekens, tussen **aanhalingstekens**

```
1  string word = "a";
2  string sentence = "Hallo!";
3  string empty = "";
```

> **TIP**
>
> `'a'` en `"a"` zien er hetzelfde uit, maar zijn **verschillende types**.

Zegt een char tegen een string: "Jij hebt makkelijk praten, jij hebt meer dan één letter."

Tekst `"1"` of getal `1`?

```
1  string oneAsText = "1";
2  int oneAsNumber = 1;
3
4  Console.WriteLine(oneAsText);
5  Console.WriteLine(oneAsNumber);
```

Output:

```
1
1
```

Op het scherm zie je hetzelfde, maar het eerste is **tekst** (een reeks tekens) en het tweede een **getal**, waarmee je kan rekenen.

😈 *`"1" + "1"` geeft `"11"`. Wiskundeleerkrachten haten deze ene truc.*

Fout gebruik compileert niet

```
1  string oneAsText = 1;    // fout
2  int oneAsNumber = "1";   // fout
```

- Regel 1: een **int**-literal toekennen aan een **string**-variabele ❌
- Regel 2: een **string**-literal toekennen aan een **int**-variabele ❌
- De compiler weigert dit, en Visual Studio onderlijnt het in het rood.



De compiler is zoals een portier: verkeerd type, niet binnen.

Even testen 🤔

Welke regels compileren?

Code	Compileert?
`string answer = "42";`	✅
`string houseNumber = 42;`	❌ een `int` in een `string`
`int age = 42;`	✅
`int score = "42";`	❌ een `string` in een `int`
`char initial = 'S';`	✅
`char grade = "A";`	❌ een `string` in een `char`

42 is het antwoord op alles. Behalve tussen aanhalingstekens, dan is het gewoon tekst.

Strings samenvoegen

Het uitgangspunt

We hebben twee variabelen:

```
1  int age = 13;
2  string name = "Finkelstein";
```

We willen een nieuwe `string result` met als inhoud:

```
Ik ben Finkelstein en ik ben 13 jaar oud.
```

Twee manieren:

1. de `+`-operator
2. stringinterpolatie met `$`

Finkelstein is al jaren 13. Niemand heeft ooit de variabele aangepast.

Manier 1: de `+`-operator

```
1  string result = "Ik ben " + name + " en ik ben " + age + " jaar oud.";
```

- Tussen strings betekent `+` niet optellen maar **aan elkaar plakken**.
- Is een stuk geen string (zoals `age`)? Dan wordt eerst de tekstvoorstelling bepaald.

> **OPMERKING**
>
> Deze manier bestaat in heel veel programmeertalen en is eenvoudig, maar ze is niet bijzonder *goed*.

Strings plakken met `+`: knutselen voor programmeurs. Lijm niet inbegrepen.

Nadelen van `+`

```
1  string result = "Ik ben " + name + " en ik ben " + age + " jaar oud.";
2  //                      ^              ^          ^      ^
3  //                      spaties aan de randen niet vergeten!
```

- Je moet steeds wisselen tussen aanhalingstekens en plustekens.
- Spaties en leestekens juist zetten is lastig.
- Een specifiek formaat (bv. aantal cijfers na de komma) vraagt veel extra werk.
- Grote, complexe strings zo opbouwen kost veel rekentijd.

👹 *Vergeet je de spaties, dan krijg je* `Ik benFinkelsteinen ik ben13jaar oud.` *Klinkt als een Duits samengesteld woord.*

Manier 2: stringinterpolatie met `$`

```
1  string result = $"Ik ben {name} en ik ben {age} jaar oud.";
```

- Het `$` **vóór de string** zegt: alles tussen `{ }` is **code**.
- `{name}` wordt vervangen door de inhoud van `name`, en `{age}` door die van `age`.
- Wat tussen de accolades staat, wordt automatisch omgezet naar tekst. Dat geldt ook voor een `int`.
- Je schrijft de string zoals hij er uiteindelijk uitziet, en dat leest veel makkelijker.

😈 *Het `$`-teken: het enige geld dat je in deze opleiding gratis krijgt.*

Naast elkaar

`+`-operator

```
1  string result = "Ik ben " + name + " en ik ben " + age + " jaar oud.";
```

Interpolatie `$`

```
1  string result = $"Ik ben {name} en ik ben {age} jaar oud.";
```

> **BELANGRIJK**
>
> Gebruik **stringinterpolatie**, tenzij anders gevraagd. Online zie je nog (vooral oudere) code die het anders doet.

Kies interpolatie. Je toekomstige zelf stuurt je een bedankkaartje. Opgesteld met interpolatie, uiteraard.

Veelgemaakte fout

```
1  string result = "Ik ben {name} en ik ben {age} jaar oud.";
2  Console.WriteLine(result);
```

```
Ik ben {name} en ik ben {age} jaar oud.
```

Het `$` ontbreekt, dus C# behandelt de accolades als gewone tekst.



Zonder `$` geen centen. Eh, geen waarden.

Omzetten van en naar strings

Het probleem

Je kan een `string` niet toekennen aan een `int` (en omgekeerd).

Toch wil je dat vaak:

```
1  Console.WriteLine("Hoe oud ben je?");
2  string input = Console.ReadLine();  // altijd een string!
3  int age = input;                    // ❌ compileert niet
```

Oplossing: de methoden van de `Convert`-klasse.



`ReadLine` geeft altijd een string. Ook als je er heel beleefd een getal in typt.

De `Convert`-klasse

```
1  int userAge = Convert.ToInt32("19");       // string → int
2  string ageAsText = Convert.ToString(19);   // int → string
```

Tussen de ronde haakjes zet je de variabele of literal die je wil omzetten.

gewenst type	methode
`int`	`ToInt32`
`double`	`ToDouble`
`string`	`ToString`
`bool`	`ToBoolean`
`byte`	`ToByte`

Een `int` krijg je dus met `ToInt32`, niet met “ToInt”. Alle conversies in de documentatie

 *`Convert`: de enige bekering die in deze les verplicht is.*

Invoer van de gebruiker verwerken

```
1  Console.WriteLine("Hoe oud ben je?");
2  string input = Console.ReadLine();
3  int age = Convert.ToInt32(input);
4  Console.WriteLine($"Volgend jaar ben je {age + 1}.");
```

```
Hoe oud ben je?
19
Volgend jaar ben je 20.
```

Of in één regel:

```
1  int age = Convert.ToInt32(Console.ReadLine());
```

Volgend jaar ben je 20. Tenzij je Finkelstein heet.

Foutloze input?

Wat als de gebruiker `IKWEEG10KG` intypt in plaats van `10,3`?

➡️ De conversie **mislukt** en je programma crasht.

> ⓘ **OPMERKING**
>
> In het begin van de leercurve ga je er **altijd** van uit dat de gebruiker foutloze input geeft. Hoe je fouten afhandelt, leer je later.

👹 *We gaan ervan uit dat de gebruiker altijd foutloze input geeft. Ja, wij lachen ook.*

Kommagetallen: punt of komma?

⚠️ **INVOER DOOR DE GEBRUIKER**

Dit hangt af van de **landinstellingen** van je besturingssysteem:

- Belgisch/Nederlands → met een **komma**: `9,81`
- Engels → met een **punt**: `9.81`

⚠️ **IN JE C#-CODE**

Daar schrijf je een `double` **altijd** met een **punt**, ongeacht je taalinstellingen.

```
1  double gravity = 9.81;         // ✅
2  double wrongGravity = 9,81;    // ❌
```

Twee schrijfwijzen voor hetzelfde getal. Typisch Belgisch: daar is vast een aparte minister voor.

Functionaliteit van strings

Length

Het **aantal symbolen** in de string:

```
1  Console.WriteLine("hallo".Length);    // 5
2  Console.WriteLine("hallo ".Length);   // 6 (de spatie telt mee!)
3  Console.WriteLine("".Length);         // 0
```

> **TIP**
>
> `Length` is een eigenschap, geen methode, en krijgt dus **geen** haakjes `()`.

De spatie telt mee. Net als dat ene vriendje dat altijd meekomt zonder uitgenodigd te zijn.

De index van een teken

Elk symbool heeft een nummer: de **index**. We beginnen te tellen bij **0**!

H	a	l	l	o	,	␣	w	e	r	e	l	d
0	1	2	3	4	5	6	7	8	9	10	11	12

- Het eerste teken heeft index `0`.
- Het laatste teken heeft index `Length - 1` (hier `13 - 1 = 12`).

😈 *Een programmeur heeft drie kinderen: kind 0, kind 1 en kind 2. Verjaardagsfeestjes zijn er erg verwarrend.*

`Substring`

Een **deel** van een string opvragen: `Substring(startindex, lengte)`

```
1  Console.WriteLine("Hallo, wereld".Substring(4, 5));  // o, we
```

H	a	l	l	o	,	␣	w	e	r	e	l	d
0	1	2	3	**4**	**5**	**6**	**7**	**8**	9	10	11	12

Laat je de lengte weg, dan loopt de substring tot het **einde**:

```
1  Console.WriteLine("Hallo, wereld".Substring(4));     // o, wereld
```

Substring werkt zoals een pizza bestellen: waar begin je, en hoeveel stukken wil je?

Strings zijn onveranderlijk ⚠️

Voer uit en verklaar: wat verschijnt er op het scherm?

A

```
string greeting = "hallo";
greeting.Substring(0, 2);
Console.WriteLine(greeting);
```

→ `hallo`

B

```
string greeting = "hallo";
Console.WriteLine(
  greeting.Substring(0, 2));
```

→ `ha`

C

```
string greeting = "hallo";
string shortGreeting =
  greeting.Substring(0, 2);
Console.WriteLine(shortGreeting);
```

→ `ha`

BELANGRIJK

Je kan een string in C# niet aanpassen. Een methode zoals `Substring` berekent een **nieuwe** string. Die moet je tonen of opslaan, anders gaat het resultaat verloren (zoals bij A).

Een string is als een koppige collega: je kan hem niet veranderen, je kan alleen een nieuwe aanwerven.

`IndexOf`

Waar komt een substring voor? Je krijgt de index van de eerste keer dat hij voorkomt.

```
1  Console.WriteLine("C# is cool".IndexOf("cool"));  // 6
2  Console.WriteLine("C# is cool".IndexOf("z"));     // -1
3  Console.WriteLine("C# is cool".IndexOf("Cool"));  // -1
```

`-1` betekent: **komt niet voor**. En let op: `IndexOf` is **hoofdlettergevoelig**.

C	#	␣	i	s	␣	c	o	o	l
0	1	2	3	4	5	**6**	7	8	9

 *`"maandagochtend".IndexOf("motivatie")` geeft `-1`.*

`ToUpper` / `ToLower`

Berekent een versie in **hoofdletters** of **kleine letters**.

```
1  Console.WriteLine("C# is cool".ToUpper());  // C# IS COOL
2  Console.WriteLine("C# is cool".ToLower());  // c# is cool
3
4  string text = "DiT iS EnGe TeKsT";
5  text.ToLower();
6  Console.WriteLine(text);       // DiT iS EnGe TeKsT (ongewijzigd!)
7  string upperText = text.ToUpper();
8  Console.WriteLine(upperText);  // DIT IS ENGE TEKST
```

 `ToUpper`: *voor als je commentaar op sociale media NOG OVERTUIGENDER moet klinken.*

`Replace`

Vervangt een substring door een andere substring.

```
1  string text = "C# is cool";
2  Console.WriteLine(text.Replace("C#", "Racket"));    // Racket is cool
3  Console.WriteLine(text.Replace("Java", "Racket"));  // C# is cool
4
5  text.Replace("C#", "Racket");
6  Console.WriteLine(text);  // C# is cool (ongewijzigd!)
```

Komt de te vervangen tekst niet voor, dan verandert er niets en krijg je ook **geen fout**.

 `"maandag".Replace("maandag", "vrijdag")` *werkt perfect. Alleen niet in je agenda.*

`TrimStart` / `TrimEnd` / `Trim`

Verwijdert **witruimte** (spaties, tabs, newlines, …) aan het begin en/of einde.

```
1  Console.WriteLine("     C# is cool".TrimStart());     // "C# is cool"
2  Console.WriteLine("C# is cool     ".TrimEnd());       // "C# is cool"
3  Console.WriteLine("    C# is cool     ".Trim());      // "C# is cool"
```

> **TIP**
>
> Handig bij invoer van de gebruiker: `Console.ReadLine().Trim()` haalt per ongeluk getypte spaties weg.

Waarom ging de string naar de kapper?

Hij moest getrimd worden.

Samengevat

Wat	Voorbeeld	Resultaat
interpolatie	`$"Ik ben {name}"`	`Ik ben Finkelstein`
conversie	`Convert.ToInt32("19")`	`19` (int)
lengte	`"hallo".Length`	`5`
deel	`"Hallo, wereld".Substring(4, 5)`	`o, we`
zoeken	`"C# is cool".IndexOf("cool")`	`6`
hoofdletters	`"C# is cool".ToUpper()`	`C# IS COOL`
vervangen	`"C# is cool".Replace("C#", "Racket")`	`Racket is cool`
witruimte weg	`"  hoi  ".Trim()`	`hoi`

BELANGRIJK

Een string verandert **nooit**. Een methode geeft altijd een **nieuwe** string terug.

Dat was het. No strings attached.

Aan de slag 💻

Maak in de klasse `StringsEnHunMethoden`:

1. **CapitalLetters**: lees tekst in en toon die in hoofdletters
2. **H3-string-interpolation**: herschrijf `MultiplicationTables` en `Space` met stringinterpolatie
3. **H3-VATCalculation**: vraag een bedrag en een btw-percentage met `Convert.ToInt32` en toon het resultaat met interpolatie

👉 Daarna: de volgende oefeningen uit de cursus (LeetSpeak, …)

😈 *Vragen? Ik zet ze om met `Convert.ToAntwoord()`. Bestaat niet. Helaas.*